



CLINICAL PROGRAMMING BOOTCAMP • MODULE 08

# Controlled Terminology

Making every study say the same thing the same way

You have already applied CT in five domains. This module explains what you were actually doing — and how it works when there are 200 values instead of 6.

*Hands-on: Notebook 10 • Applying Controlled Terminology (SAS)*

# By the end of this module you can...

1

**Say what a codelist is** and where the authoritative version comes from.

2

**Tell extensible from non-extensible** and know which one you may add a value to.

3

**Recognise the four kinds of CT work** decode, normalise, rename, derive — you have done all four already.

4

**Write mapping code that fails loudly** instead of silently blanking what it does not recognise.

5

**Explain CT versioning** why a study pins a dated release and never drifts.

# Six sites, one concept, six spellings

## Without controlled terminology

"Mild"

"mild"

"MILD"

"1 - Mild"

"Grade 1"

"leichte"

*Six values. One concept. No way to count them together.*

## With controlled terminology

**MILD**

One submission value, defined by CDISC, spelled and cased exactly this way in every study, by every sponsor, worldwide.

## Why a regulator cares

A reviewer assessing a safety signal wants every severe event across every study of this drug class — dozens of submissions from different sponsors. That query only works if all of them wrote SEVERE the same way. CT is what makes your study POOLABLE with everyone else's.

YOU HAVE ALREADY DONE THIS

# Every coded variable in ABC-01 — and all eight changed

| Variable | Domain | Raw values as collected  | SDTM submission values   | Kind      |
|----------|--------|--------------------------|--------------------------|-----------|
| SEX      | DM     | 1 · 2                    | M · F                    | decode    |
| RACE     | DM     | White · asian · 'White ' | WHITE · ASIAN · ...      | normalise |
| ETHNIC   | DM     | Not Hispanic or Latino   | NOT HISPANIC OR LATINO   | normalise |
| AESEV    | AE     | mild · Mild · severe     | MILD · MODERATE · SEVERE | normalise |
| AESER    | AE     | No · N · Yes             | N · Y                    | decode    |
| AEREL    | AE     | Possibly related         | POSSIBLY RELATED         | normalise |
| AEOUT    | AE     | 1 · 2 · 3                | RECOVERED/RESOLVED · ... | decode    |
| LBTEST   | LB     | White Blood Cells        | Leukocytes               | RENAME    |

## Not one of the eight passes through unchanged.

That is the normal state of affairs: raw data is what the site could type, SDTM is what the standard requires. The last row is the one that bites — five of six lab tests map to themselves, so a programmer who spot-checks two rows concludes the column needs no mapping at all, and ships "White Blood Cells" into a submission.

## WHAT A CODELIST ACTUALLY IS

# Four things, and only one of them goes in your dataset

### NCI code

C66731

The permanent identifier for the CODELIST itself. Never changes, even if the name does.

### Submission value

MILD

The string you put IN THE DATASET. Exact spelling, exact case.

### Preferred term

Mild Adverse Event

The human-readable name. Documentation only — never submitted.

### Definition

"An adverse event..."

What the term means. Settles arguments about which value applies.

*Only the SUBMISSION VALUE goes in the dataset. The rest is metadata — it lives in define.xml, which is how a reviewer knows which codelist a variable draws from.*

# May you add a value of your own?

## NON-EXTENSIBLE

The list is **CLOSED**. You may use only the values CDISC published.

Examples in ABC-01

**SEX** M · F · U · UNDIFFERENTIATED

**NY (AESER)** N · Y · NA · U

**AESEV** MILD · MODERATE · SEVERE

A value outside the list is a **CONFORMANCE ERROR**. There is no "our study uses Grade 1" — you map to MILD or you fail validation.

## EXTENSIBLE

The list is a **STARTING POINT**. You may add a study-specific value.

Examples in ABC-01

**RACE** WHITE · ASIAN · ...

**LBTEST** hundreds of tests

**UNIT** mg · kg · % · IU

But adding is a **LAST RESORT**. Search the published list properly first — most "we need a new value" turns out to be a term that already exists under a different name.

*How do you know which one a codelist is? The CT release says so, and define.xml records it per variable. Never guess — check.*

## FOUR KINDS OF WORK

# Every CT mapping you will ever write is one of these

### DECODE

SEX 1 -> M      AEOUT 3 -> NOT RECOVERED/NOT RESOLVED

A code stands for a term

The raw value is meaningless on its own. You NEED the data dictionary.

### NORMALISE

'mild' -> MILD      'White ' -> WHITE

Right term, wrong shape

Trim and upper-case. Easy — and the easiest to do carelessly.

### RENAME

'White Blood Cells' -> Leukocytes

Different term entirely

Only a lookup finds these. No string rule will ever produce them.

### DERIVE

AESTDY >= 1 -> AETRTEM = Y

Not collected at all

The value is computed from other variables, not mapped from one.

*The danger rises down the list. NORMALISE looks trivial, so people stop checking — and RENAME hides inside a column that mostly normalises cleanly.*

# What does your code do with a value it has never seen?

## What you have been writing

```
select (strip(aeout));  
  when ("1") aeout_c = "RECOVERED/RESOLVED";  
  when ("2") aeout_c = "RECOVERING/RESOLVING";  
  ...  
  otherwise aeout_c = "";  
end;
```

## Silent blanking

A new code 6 appears in next month's extract. Your program turns it into a blank and reports success. The dataset looks CLEANER than one with a visible problem — and the information is gone.

Worst of both worlds.

## What production code writes

```
select (strip(aeout));  
  when ("1") aeout_c = "RECOVERED/RESOLVED";  
  when ("2") aeout_c = "RECOVERING/RESOLVING";  
  ...  
  otherwise do;  
    aeout_c = "";  
    put "ERROR: unmapped AEOUT >" aeout  
      "< for " usubjid=;  
  end;
```

## Failing loudly

The same code 6 writes ERROR to the log with the subject id. Someone reads it, asks the data manager, and finds out the CRF gained an option nobody told you about.

A new value is a SPEC CHANGE, not something a mapping program should absorb.



# The standard moves; your study must not

CDISC publishes Controlled Terminology QUARTERLY. Terms are added, definitions clarified, occasionally a term is retired.

2023-12-15

release

2024-03-29

◀ ABC-01 pins this

2024-06-28

release

2024-09-27

release

## A study pins ONE dated release and uses it for the study's whole life.

The version is recorded in define.xml. A trial that ran for four years was mapped against the CT release chosen at the start — not whatever came out last quarter. Changing mid-study would make early and late data incomparable, which is exactly what CT exists to prevent.

## Where the authoritative list lives

NCI Enterprise Vocabulary Services (NCI-EVS) publishes CDISC CT as downloadable files — text, Excel and OWL. That download is the source of truth. Not a colleague's spreadsheet, not last study's format catalogue, and not this deck: those are all copies, and copies go stale.

# The method has to change

## What you have done so far

A hard-coded SELECT with one WHEN per value.

Perfectly fine for 6 values. Unreadable and unmaintainable at 200, and impossible to reconcile against a published list.

## What production does

Keep the mapping as DATA, not code — a lookup dataset with one row per raw value.

Then join. The mapping becomes something you can print, review, diff and sign off.

## SAS · data-driven CT

```
/* the mapping is a DATASET, not a hundred WHEN clauses */
proc format cntlin = ct_lookup; /* formats built from the list */
run;
aesev = put(strip(upcase(aesev_raw)), $sevmap.);
/* anything the lookup missed is REPORTED, never blanked */
proc sql;
    select distinct aesev_raw from raw
    where put(strip(upcase(aesev_raw)), $sevmap.) = ""; /* 0 rows */
quit;
```

# Sponsor-defined terminology

**AEREL is the example sitting in your own data.**

CDISC does not publish a relationship-to-study-treatment codelist, because sponsors genuinely differ: some use 2 categories, some 4, some 5. ABC-01 uses four — RELATED, POSSIBLY RELATED, UNLIKELY RELATED, NOT RELATED — and that choice comes from the PROTOCOL, not from CDISC.

|                        | CDISC codelist     | Sponsor-defined                             |
|------------------------|--------------------|---------------------------------------------|
| Who decides the values | CDISC / NCI-EVS    | the sponsor, in the protocol                |
| Where it is documented | the CT release     | <b>**define.xml**</b> — you must publish it |
| May you invent a value | only if extensible | yes, but define it up front                 |
| Validation checks it   | yes, automatically | only against your own define.xml            |

**The obligation is the same.** Sponsor-defined does not mean informal. The list must be fixed before data comes in, applied consistently, and published in define.xml — otherwise a reviewer has no way to know what "UNLIKELY RELATED" meant in your study.

# The five that cost the most

1

## Spot-checking a column

Five of six lab tests map to themselves. The sixth is Leukocytes.

→ check every value, every time

2

## Blanking the unrecognised

A new code becomes a blank and the run reports success.

→ write ERROR to the log

3

## Extending a closed codelist

"Our study needs Grade 1" — SEX, AESEV and NY are non-extensible.

→ map to the published value

4

## Using the latest CT mid-study

Early and late data stop being comparable.

→ pin the dated release

5

## Copying a format catalogue

Last study's catalogue was built from an older CT release.

→ rebuild from the NCI-EVS download